UNIVERSITATEA TEHNICĂ A MOLDOVEI
FACULTATEA CALCULATOARE, INFORMATICĂ ȘI MICROELECTRONICĂ
DEPARTAMENTUL INGINERIA SOFTWARE ȘI AUTOMATICĂ

Lucrare de laborator Nr. 4

Tehnici de programare

Python Fundamentals

Realizat: Miron Cezar

st. gr. SI-262

Chișinău, 2026

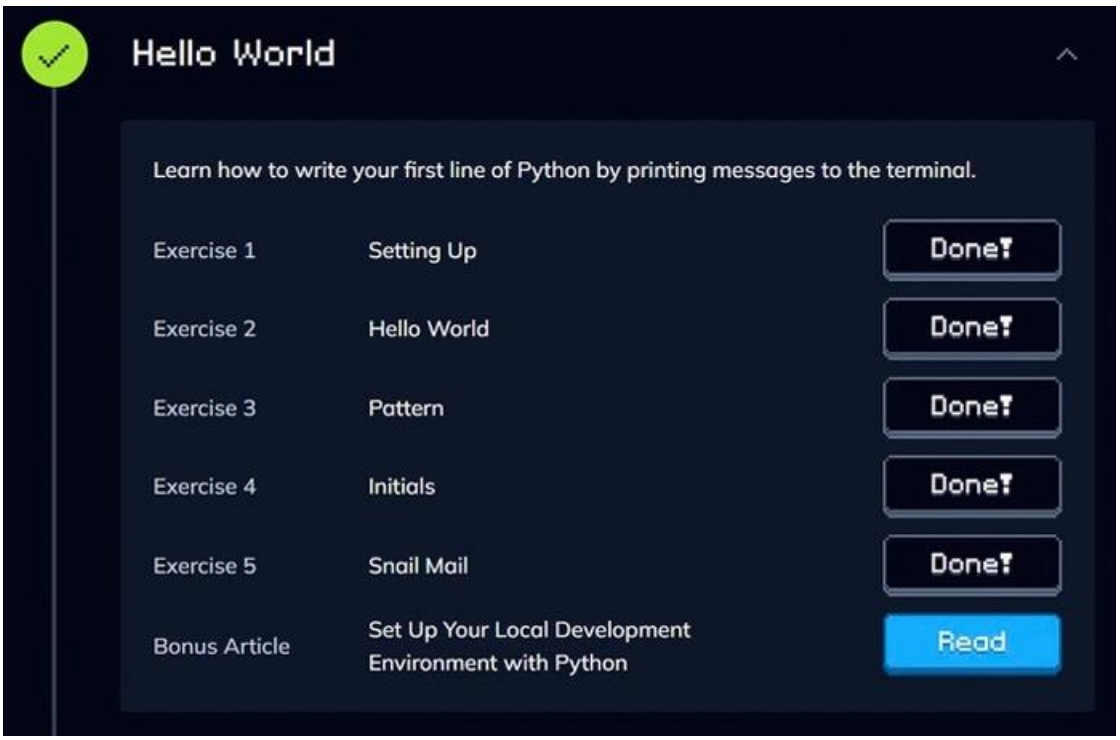
SARCINA LUCRĂRII DE LABORATOR

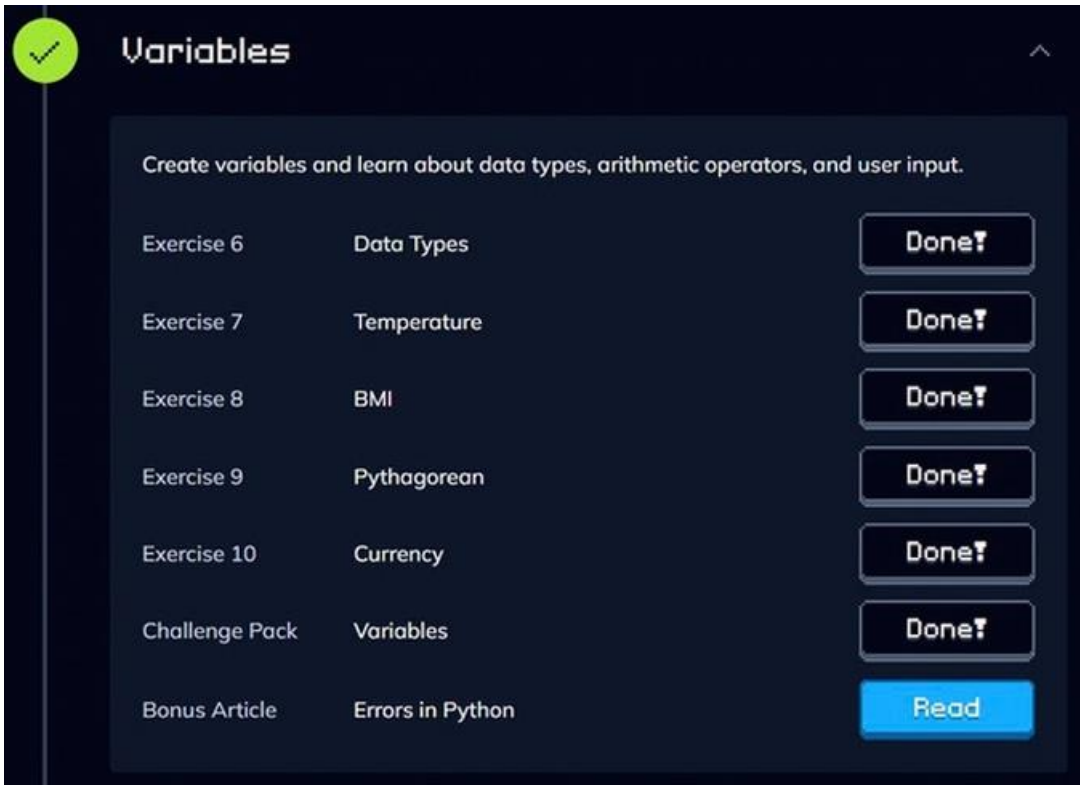
Completarea cursului de pe platforma CODEDEX

Completarea cursului de pe platforma learnpython.org

DESFĂȘURAREA LUCRĂRII

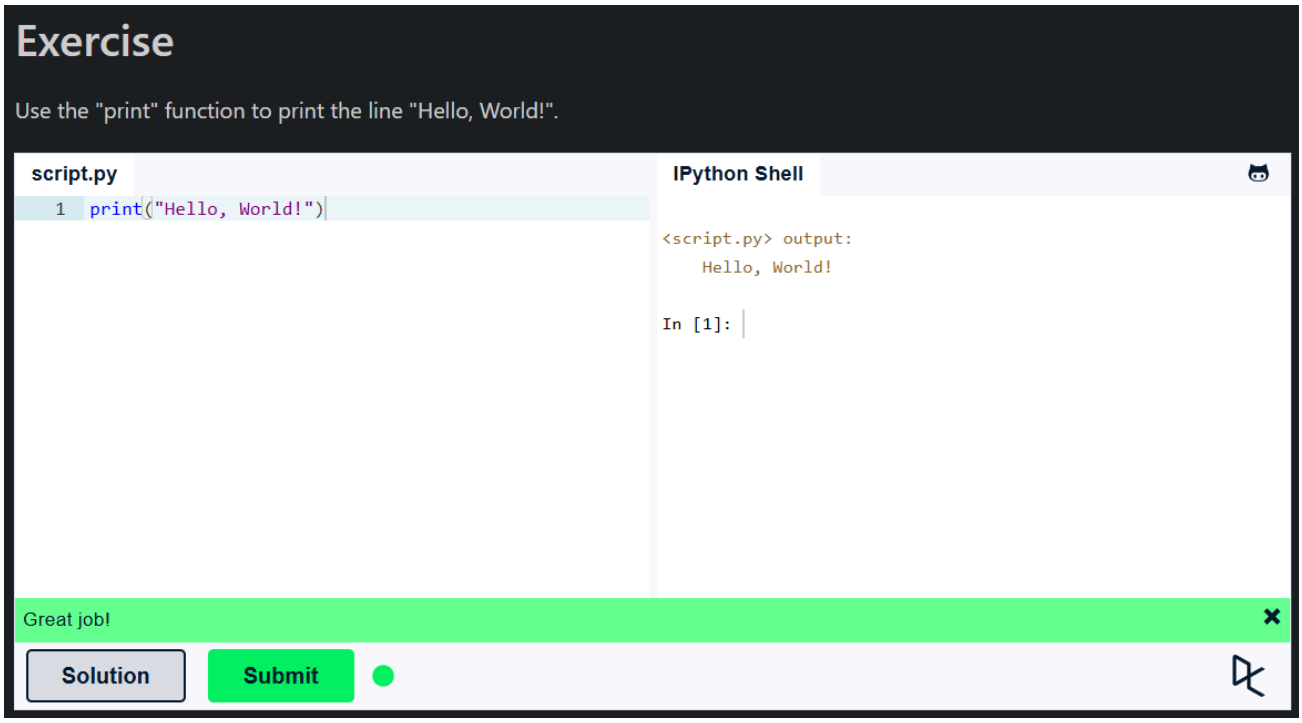
1. Codedex



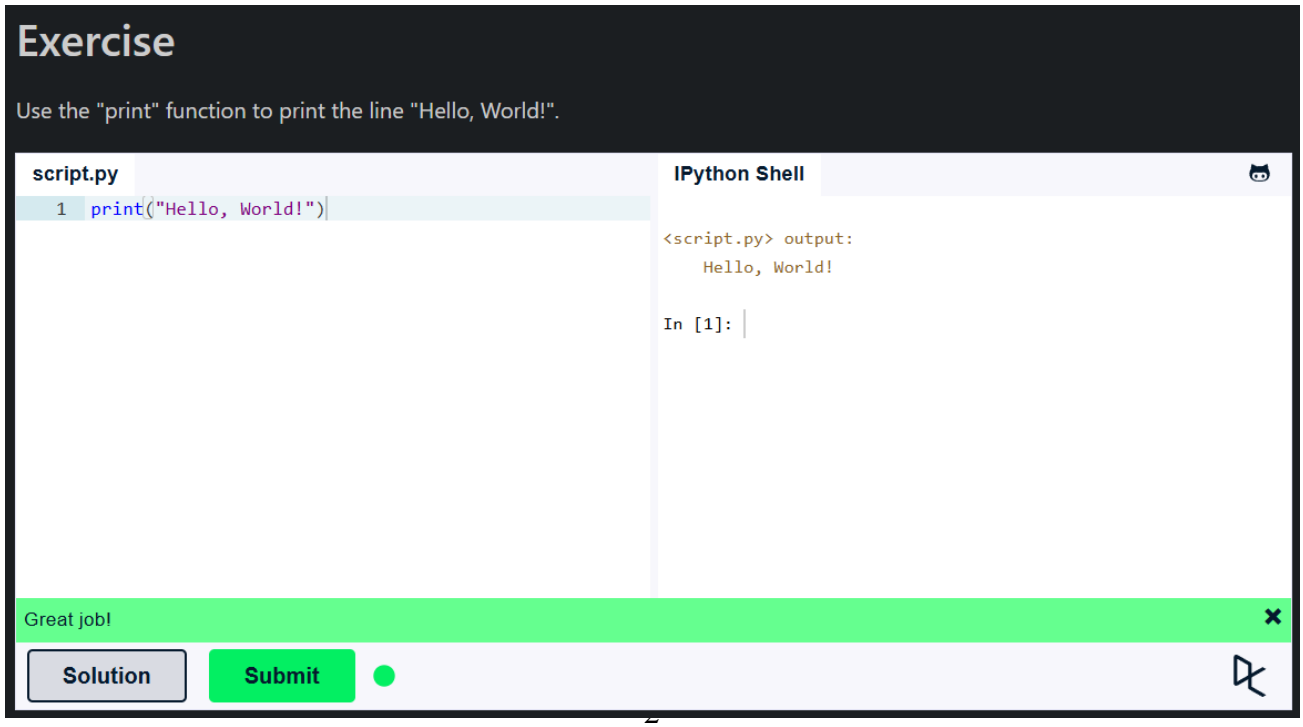


2. learnpython.org

HELLO WORLD



Variables and Types



Lists

Exercise

In this exercise, you will need to add numbers and strings to the correct lists using the "append" list method. You must add the numbers 1,2, and 3 to the "numbers" list, and the words 'hello' and 'world' to the strings variable.

You will also have to fill in the variable second_name with the second name in the names list, using the brackets operator []. Note that the index is zero-based, so if you want to access the second item in the list, its index will be 1.

script.py

```
1  numbers = [1,2,3]
2  strings = ['hello','world']
3  names = ["John", "Eric", "Jessica"]
4
5  # write your code here
6  second_name = names[1]
7
8
9  # this code should write out the filled arrays and
   the second name in the names list (Eric).
10 print(numbers)
11 print(strings)
12 print("The second name on the names list is %s" %
   second_name)
```

IPython Shell

```
<script.py> output:
    [1, 2, 3]
    ['hello', 'world']
    The second name on the names list is Eric

In [1]:
```

Great Job!

Solution | Submit

Basic Operators

Exercise

The target of this exercise is to create two lists called x_list and y_list, which contain 10 instances of the variables x and y, respectively. You are also required to create a list called big_list, which contains the variables x and y, 10 times each, by concatenating the two lists you have created.

script.py

```
1  x = object()
2  y = object()
3
4  # TODO: change this code
5  x_list = [x] * 10
6  y_list = [y] * 10
7  big_list = x_list + y_list
8
9  print("x_list contains %d objects" % len(x_list))
10 print("y_list contains %d objects" % len(y_list))
11 print("big_list contains %d objects" % len
   (big_list))
12
13 # testing code
14 if x_list.count(x) == 10 and y_list.count(y) ==
   10:
```

IPython Shell

```
    y_list contains 10 objects
    big_list contains 1 objects
    Almost there...

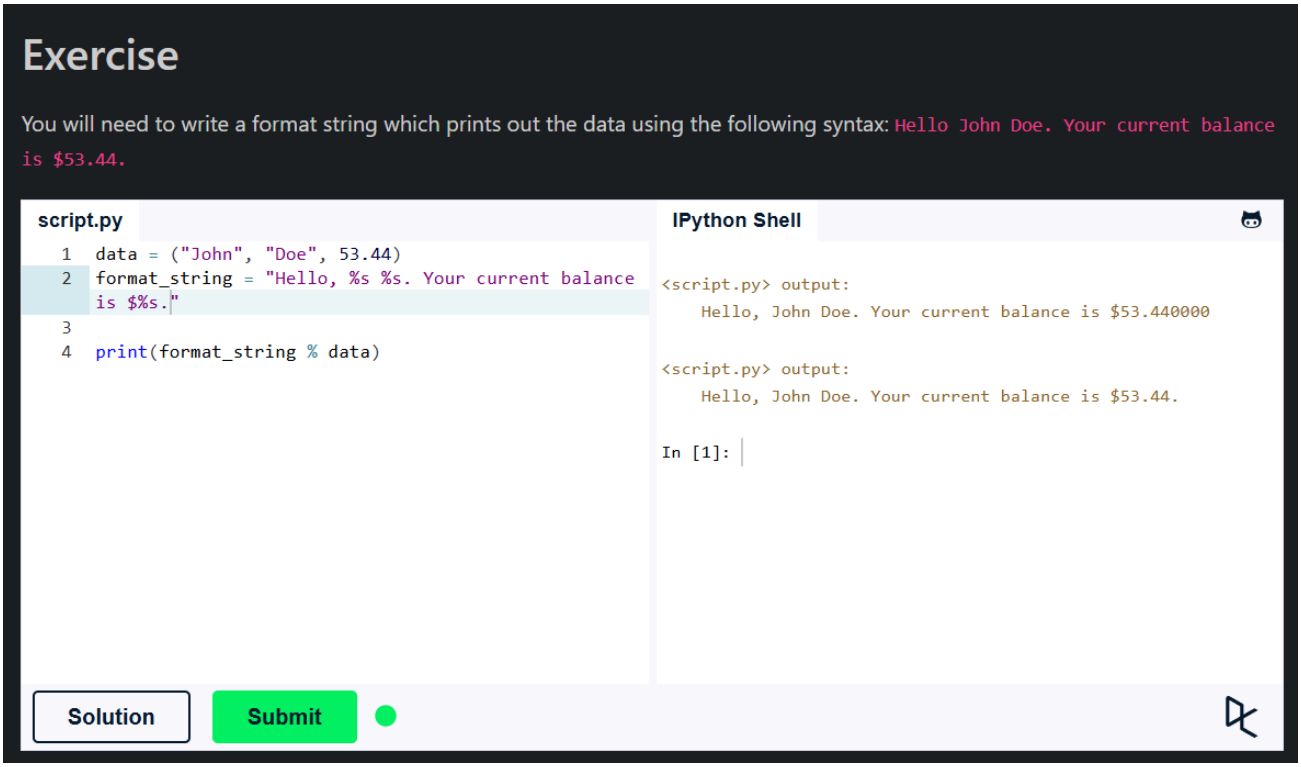
<script.py> output:
    x_list contains 10 objects
    y_list contains 10 objects
    big_list contains 20 objects
    Almost there...
    Great!

<script.py> output:
    x_list contains 10 objects
    y_list contains 10 objects
```

Good work!

Solution | Submit

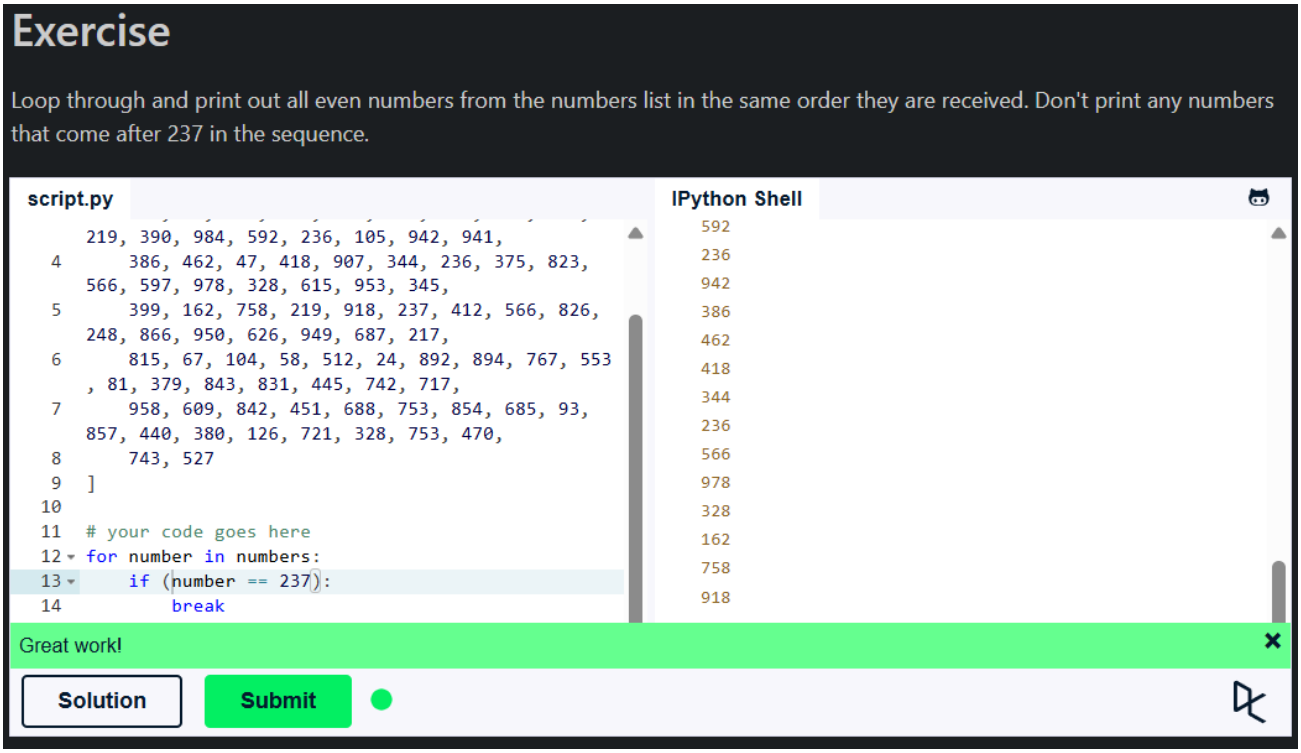
String Formatting



Basic String Operations

Conditions

Loops


Exercise
Loop through and print out all even numbers from the numbers list in the same order they are received. Don't print any numbers that come after 237 in the sequence.
script.py
219, 390, 984, 592, 236, 105, 942, 941,
386, 462, 47, 418, 907, 344, 236, 375, 823,
566, 597, 978, 328, 615, 953, 345,
399, 162, 758, 219, 918, 237, 412, 566, 826,
248, 866, 950, 626, 949, 687, 217,
815, 67, 104, 58, 512, 24, 892, 894, 767, 553
, 81, 379, 843, 831, 445, 742, 717,
958, 609, 842, 451, 688, 753, 854, 685, 93,
857, 440, 380, 126, 721, 328, 753, 470,
743, 527
]
your code goes here
for number in numbers:
if (number == 237):
break
IPython Shell
592
236
942
386
462
418
344
236
566
978
328
162
758
918
Great work!
Solution
Submit

Functions

Exercise

In this exercise you'll use an existing function, and while adding your own to create a fully functional program.

1. Add a function named `list_benefits()` that returns the following list of strings: "More organized code", "More readable code", "Easier code reuse", "Allowing programmers to share and connect code together"
2. Add a function named `build_sentence(info)` which receives a single argument containing a string and returns a sentence starting with the given string and ending with the string " is a benefit of functions!"
3. Run and see all the functions work together!

script.py

```
# Modify this function to return a list of strings as defined above
def list_benefits():
    return ["More organized code", "More readable code", "Easier code reuse", "Allowing programmers to share and connect code together"]

# Modify this function to concatenate to each benefit - " is a benefit of functions!"
def build_sentence(benefit):
    return benefit + " is a benefit of functions!"

def name_the_benefits_of_functions():
    list_of_benefits = list_benefits()
    for benefit in list_of_benefits:
        print(build_sentence(benefit))
```

IPython Shell

```
<script.py> output:
    More organized code is a benefit of functions!
    More readable code is a benefit of functions!
    Easier code reuse is a benefit of functions!
    Allowing programmers to share and connect code together
        is a benefit of functions!

<script.py> output:
    More organized code is a benefit of functions!
    More readable code is a benefit of functions!
    Easier code reuse is a benefit of functions!
    Allowing programmers to share and connect code together
        is a benefit of functions!
```

Nice work!

Solution Submit

Classes and Objects

Exercise

We have a class defined for vehicles. Create two new vehicles called car1 and car2. Set car1 to be a red convertible worth $60,000.00 with a name of Fer, and car2 to be a blue van named Jump worth $10,000.00.

script.py

```
        desc_str = "%s is a %s %s worth $%.2f." % (self.name, self.color, self.kind, self.value)
        return desc_str
# your code goes here
car1 = Vehicle()
car1.name = "Fer"
car1.color = "red"
car1.kind = "convertible"
car1.value = 60000

car2 = Vehicle()
car2.name = "Jump"
car2.color = "blue"
car2.kind = "van"
car2.value = 10000
# test code
```

IPython Shell

```
<script.py> output:
    Fer is a red convertible worth $60000.00.
    Jump is a blue van worth $10000.00.

In [1]:
```

Great job!

Solution Submit

Dictionaries

Exercise

Add "Jake" to the phonebook with the phone number 938273443, and remove Jill from the phonebook.

script.py

```
phonebook = {
    "John" : 938477566,
    "Jack" : 938377264,
    "Jill" : 947662781
}
# your code goes here
phonebook["Jake"] = 938273443
phonebook.pop("Jill")
# testing code
if "Jake" in phonebook:
    print("Jake is listed in the phonebook.")

if "Jill" not in phonebook:
    print("Jill is not listed in the phonebook.")
```

IPython Shell

```
<script.py> output:
    Jake is listed in the phonebook.
    Jill is not listed in the phonebook.

In [1]:
```

Nice work!

Solution Submit

Modules and Packages

Exercise

In this exercise, print an alphabetically sorted list of all the functions in the `re` module containing the word `find`.

script.py

```
import re

# Your code goes here
find_members = []
for member in dir(re):
    if "find" in member:
        find_members.append(member)

print(sorted(find_members))
```

IPython Shell

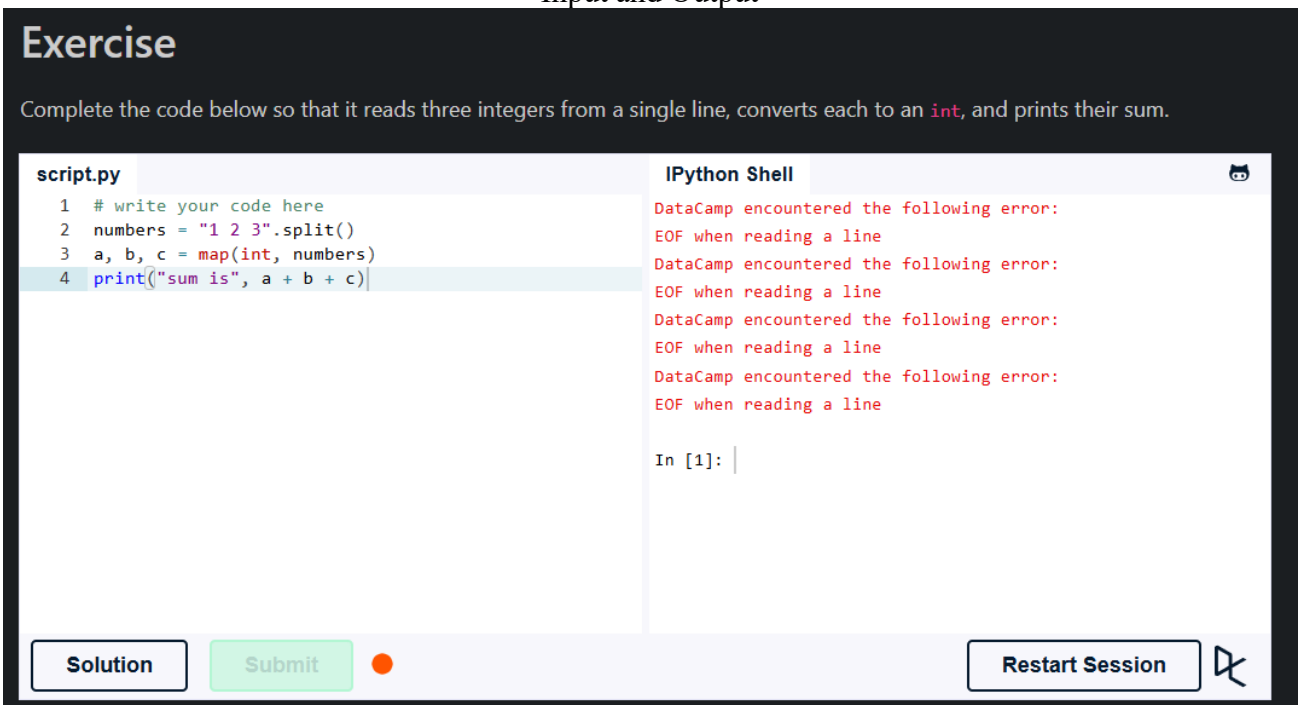
```
<script.py> output:
    ['findall', 'finditer']

In [1]:
```

Great work!

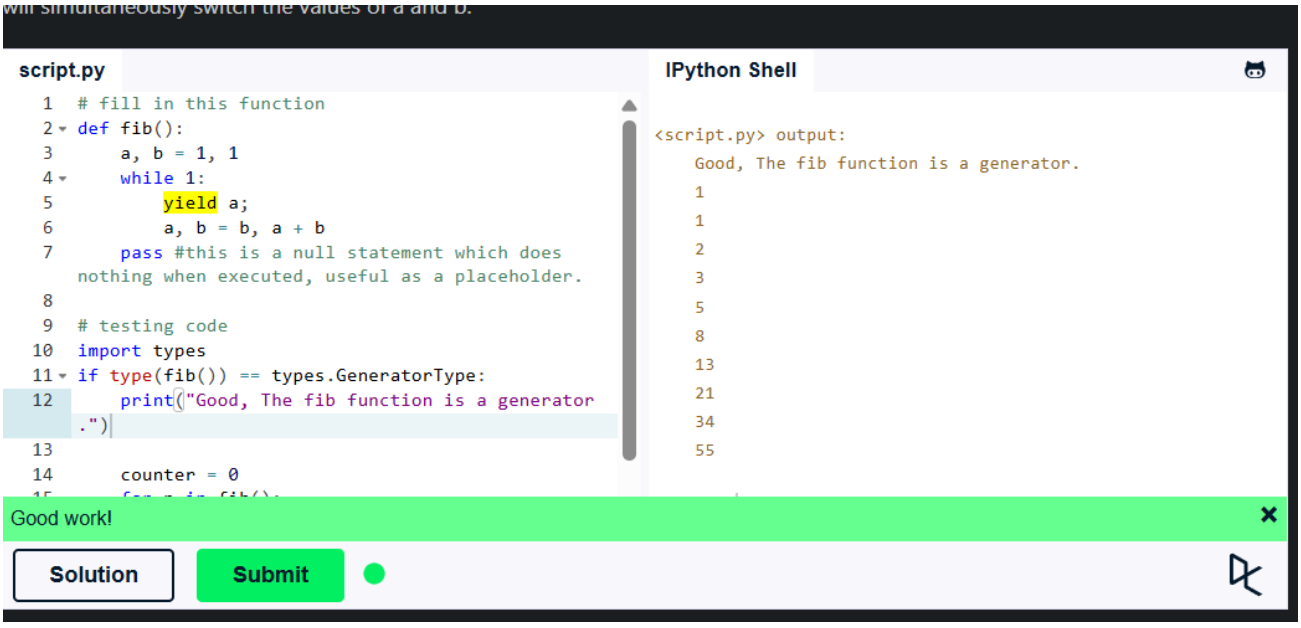
Solution Submit

Input and Output



Advanced Tutorials

Generators



List Comprehensions

Exercise

Using a list comprehension, create a new list called "newlist" out of the list "numbers", which contains only the positive numbers from the list, as integers.

script.py

```
1  numbers = [34.6, -203.4, 44.9, 68.3, -12.2, 44.6,
   12.7]
2  newlist = [int (number) for number in numbers if
   number > 0]
3  print(newlist)
```

IPython Shell

```
Traceback (most recent call last):
  File "script.py", line 2, in <module>
    newlist = [int (numbre) for number in numbers if number >
        0]
  File "script.py", line 2, in <listcomp>
    newlist = [int (numbre) for number in numbers if number >
        0]
NameError: name 'numbre' is not defined

<script.py> output:
    [34, 44, 68, 44, 12]

In [1]:
```

Very nice!

Solution Submit

Lambda Functions

Exercise

Write a program using lambda functions to check if a number in the given list is odd. Print "True" if the number is odd or "False" if not for each element.

script.py

```
1  l = [2,4,7,3,14,19]
2  for i in l:
3      # your code here
4      isodd = lambda number : number % 2 == 1
5      print(isodd(i))
```

IPython Shell

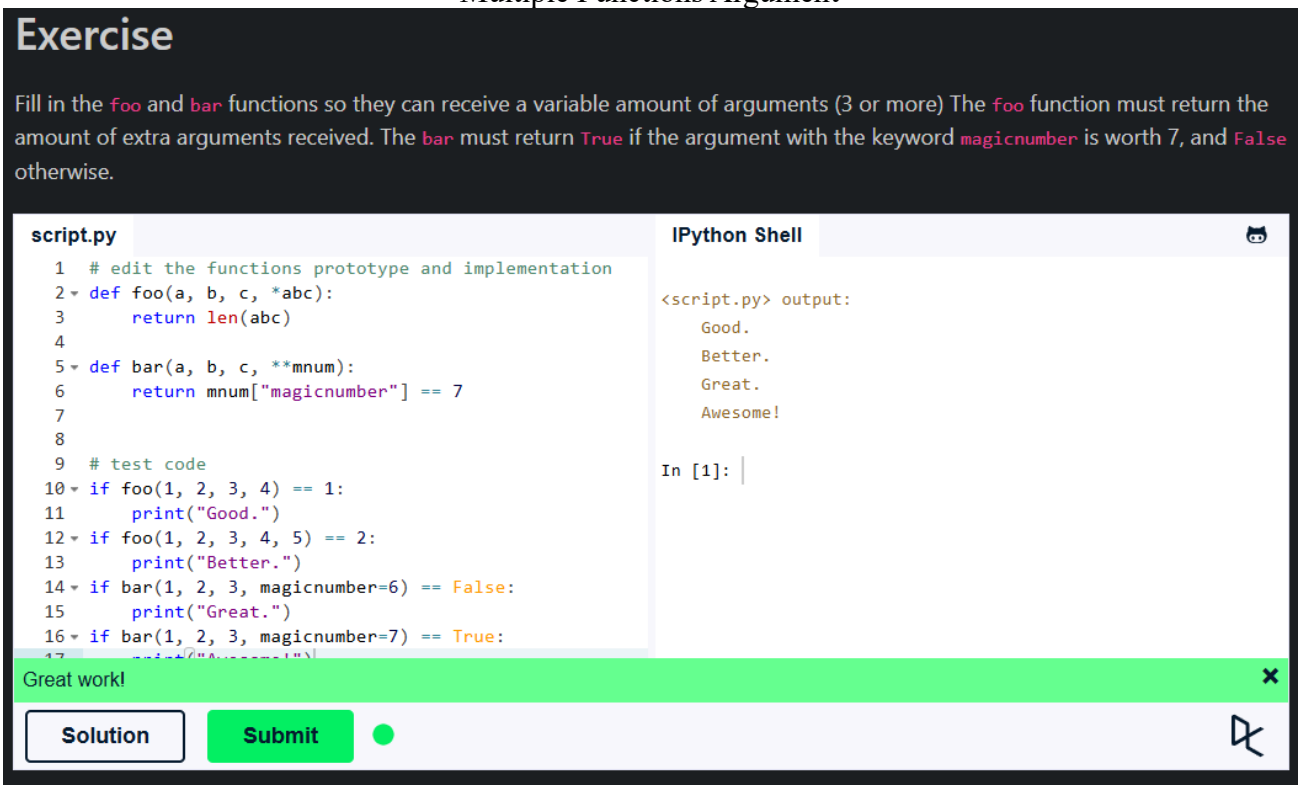
```
<script.py> output:
    False
    False
    True
    True
    False
    True

In [1]:
```

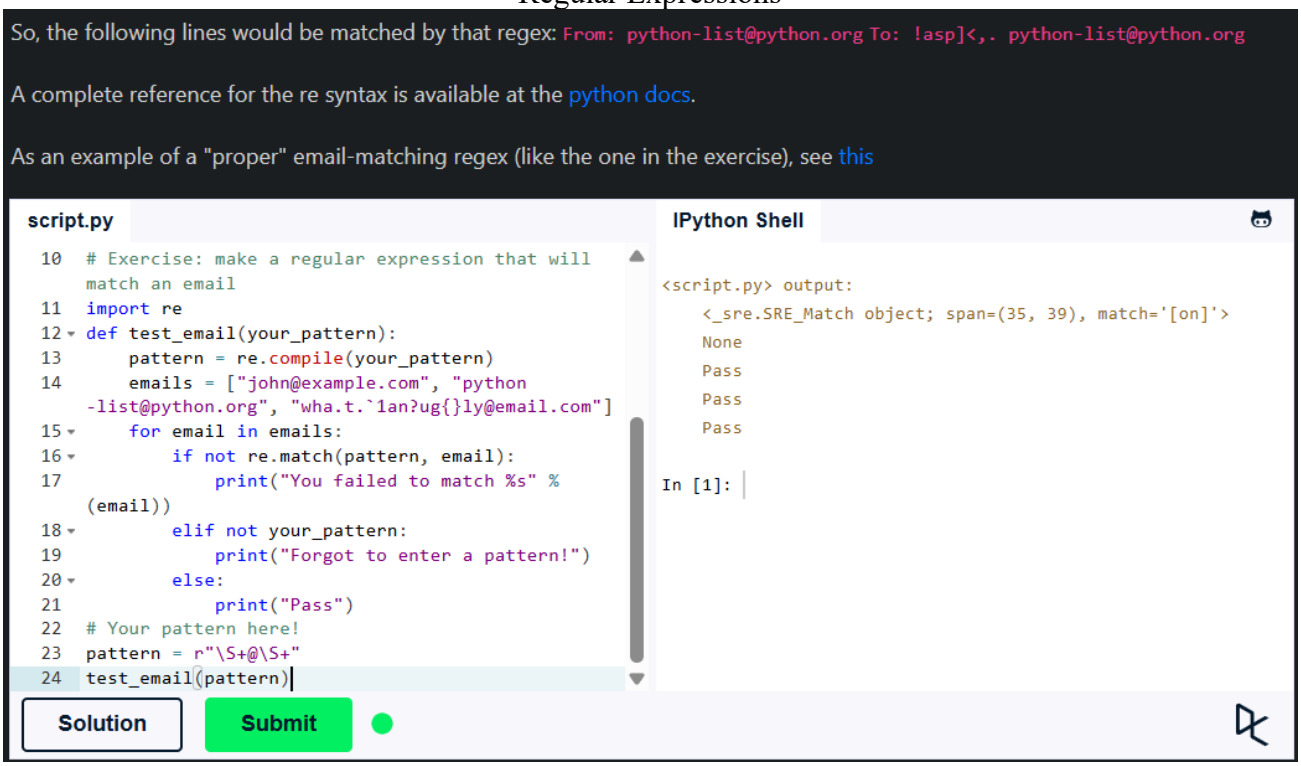
Nice work!

Solution Submit

Multiple Functions Argument



Regular Expressions



Exception Handling

Exercise

Handle all the exception! Think back to the previous lessons to return the last name of the actor.

script.py

```
1  # Setup
2  actor = {"name": "John Cleese", "rank": "awesome"}
3
4  # Function to modify!!!
5  def get_last_name():
6      return actor["name"].split()[1]
7
8  # Test code
9  get_last_name()
10 print("All exceptions caught! Good job!")
11 print("The actor's last name is %s" % get_last_name
   ())
```

IPython Shell

```
Traceback (most recent call last):
  File "script.py", line 9, in <module>
    get_last_name()
  File "script.py", line 6, in get_last_name
    return actor["last_name"].split()[1]
KeyError: 'last_name'

<script.py> output:
    All exceptions caught! Good job!
    The actor's last name is Cleese

In [1]:
```

Great work!

Solution Submit

Sets

In the exercise below, use the given lists to print out a set containing all the participants from event A which did not attend event B.

script.py

```
1  a = ["Jake", "John", "Eric"]
2  b = ["John", "Jill"]
3  A = set(a)
4  B = set(b)
5  print(A.difference(B))
```

IPython Shell

```
<script.py> output:
    {'Jake', 'Eric'}

In [1]:
```

Nice work!

Solution Submit

Serialization

The aim of this exercise is to print out the JSON string with key-value pair "Me" : 800 added to it.

script.py

```
1  import json
2
3  # fix this function, so it adds the given name
4  # and salary pair to salaries_json, and return it
5  def add_employee(salaries_json, name, salary):
6      salaries = json.loads(salaries_json)
7      # Add your code here
8      salaries[name] = salary
9      return json.dumps(salaries)
10
11 # test code
12 salaries = '{"Alfred" : 300, "Jane" : 400 }'
13 new_salaries = add_employee(salaries, "Me", 800)
14 decoded_salaries = json.loads(new_salaries)
15 print(decoded_salaries["Alfred"])
16 print(decoded_salaries["Jane"])
```

IPython Shell

```
<script.py> output:
    300
    400
    800

In [1]:
```

Great work!

Solution Submit

Partial Functions

Exercise

Edit the function provided by calling partial() and replacing the first three variables in func(). Then print with the new partial function using only one input variable so that the output equals 60.

script.py

```
1  #Following is the exercise, function provided:
2  from functools import partial
3  def func(u, v, w, x):
4      return u*4 + v*3 + w*2 + x
5  #Enter your code here to create and print with your
   partial function
6  tst = partial(func, 5, 6, 7)
7  print(tst(8))
```

IPython Shell

```
<script.py> output:
    60

In [1]:
```

Good job!

Solution Submit

Concluzie

Parcurgând materialele de pe Codexdex și learnpython.org, mi-am format o fundație solidă în programare. Am trecut de la concepte de bază – precum variabile, structuri decizionale și bucle – până la tehnici avansate, cum ar fi funcțiile lambda, generatorii și tratarea erorilor. Îmbinarea teoriei cu aplicațiile practice și exercițiile interactive mi-a permis să înțeleg în profunzime limbajul Python și să-mi dezvolt abilitățile de codare.